# hwk2_Yiyi_Zhu

## Part 1

### 1,1

### (a)

The intercept of 37.227 represents the theoretical expected mpg when both vehicle weight and horsepower are zero, while controlling for horsepower, each 1,000-pound increase in vehicle weight (wt) significantly decreases expected fuel efficiency by approximately 3.878 mpg (p = 1.12e-06), and holding weight constant, every one-unit increase in horsepower (hp) significantly reduces the expected mpg by about 0.032 (p = 0.00145).

```
model1 <- lm(mpg ~ wt + hp, data = mtcars)
summary(model1)
```

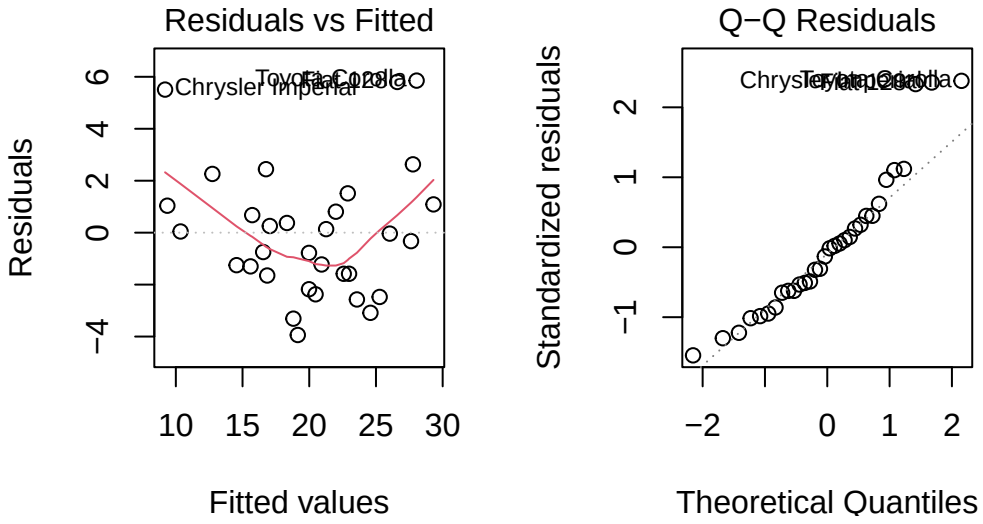
```
Call:
lm(formula = mpg ~ wt + hp, data = mtcars)

Residuals:
   Min     1Q Median     3Q    Max
-3.941 -1.600 -0.182  1.050  5.854

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) 37.22727    1.59879  23.285  < 2e-16 ***
wt          -3.87783    0.63273  -6.129 1.12e-06 ***
hp          -0.03177    0.00903  -3.519  0.00145 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
Residual standard error: 2.593 on 29 degrees of freedom
Multiple R-squared:  0.8268,    Adjusted R-squared:  0.8148
F-statistic: 69.21 on 2 and 29 DF,  p-value: 9.109e-12
```

**(b)**

The Residuals vs Fitted plot reveals a slight U-shaped curvature in the red smoothing line, suggesting a minor non-linear relationship that the current additive model does not fully capture, while the Q-Q plot shows most standardized residuals following the normal reference line reasonably well, with only slight deviations at the extreme tails for specific observations like the Chrysler Imperial and Toyota Corolla, indicating the normality assumption is adequately satisfied for standard inference.

```r
par(mfrow = c(1, 2))
plot(model1, which = 1)
plot(model1, which = 2)
```




**(c)**

The variance inflation factors (VIF) for both wt and hp are exactly 1.766625, which falls well below the standard diagnostic threshold of 5, indicating that there is no severe multicollinearity between vehicle weight and horsepower and ensuring that the standard errors of the coefficient estimates remain reliable.

```r
library(car)
```

```
Loading required package: carData
```

```r
vif(model1)
```

```
      wt       hp
1.766625 1.766625
```

### (d)

I prefer the interaction model because the ANOVA comparison yields an F-statistic of 14.088 with a highly significant p-value of 0.0008108, demonstrating that including the interaction term (wt:hp) significantly reduces the residual sum of squares from 195.05 to 129.76, which confirms that the multiplicative term provides a substantially better fit to the data by successfully capturing how the negative effect of weight on fuel economy is dependent on the engine's horsepower.

```r
model_int <- lm(mpg ~ wt * hp, data = mtcars)
anova(model1, model_int)
```

```
Analysis of Variance Table

Model 1: mpg ~ wt + hp
Model 2: mpg ~ wt * hp
  Res.Df    RSS Df Sum of Sq      F    Pr(>F)
1     29 195.05
2     28 129.76  1    65.286 14.088 0.0008108 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

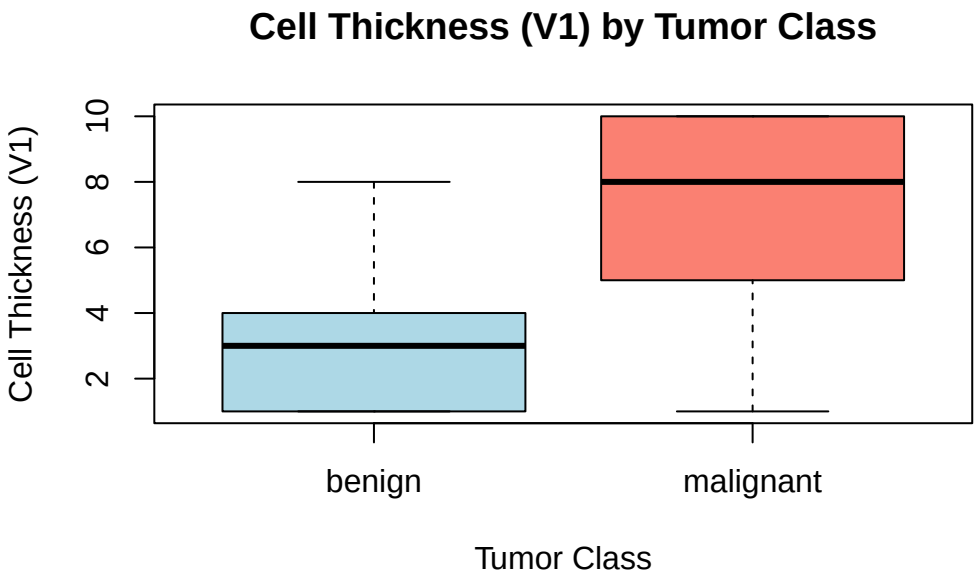
## 1.2

### (a)

The boxplot demonstrates a clear distinction in cell thickness (V1) between the two tumor classes. Benign tumors exhibit a significantly lower median cell thickness (around 3) with a narrow interquartile range concentrated between 1 and 4. In contrast, malignant tumors show a much higher median thickness of 8, with the middle 50% of the data widely spread between

5 and 10. This pronounced visual separation indicates that elevated cell thickness is a strong predictor of malignancy

```r
data(biopsy, package = "MASS")
biopsy <- na.omit(biopsy)

biopsy$class <- factor(biopsy$class,
                       levels = c("benign", "malignant"))

boxplot(V1 ~ class, data = biopsy,
        main = "Cell Thickness (V1) by Tumor Class",
        xlab = "Tumor Class", ylab = "Cell Thickness (V1)",
        col = c("lightblue", "salmon"))
```




**(b)**

The logistic regression summary shows that V1, V2, and V3 are all highly significant positive predictors of malignancy ($p < 0.001$). Based on the calculated odds ratios, holding other variables constant, a one-unit increase in cell thickness (V1) increases the odds of the tumor being malignant by approximately 80.7% (OR $\approx 1.807$). Similarly, one-unit increases in cell size uniformity (V2) and cell shape uniformity (V3) multiply the odds of malignancy by factors of 1.895 and 2.063, respectively.

```r
model_log <- glm(class ~ V1 + V2 + V3, data = biopsy, family = binomial)


summary(model_log)
```

```
Call:
glm(formula = class ~ V1 + V2 + V3, family = binomial, data = biopsy)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)  -7.7210     0.6969 -11.079  < 2e-16 ***
V1            0.5918     0.1030   5.746 9.14e-09 ***
V2            0.6390     0.1704   3.751 0.000176 ***
V3            0.7240     0.1661   4.358 1.31e-05 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 884.35  on 682  degrees of freedom
Residual deviance: 176.50  on 679  degrees of freedom
AIC: 184.5

Number of Fisher Scoring iterations: 7
```

```r
exp(coef(model_log))
```

```
 (Intercept)           V1           V2           V3
0.0004433959 1.8073247714 1.8946201232 2.0627331079
```

### (c)

Based on the confusion matrix using a 0.5 classification threshold, the model achieves a high overall accuracy of 95.02%. It correctly identifies 91.63% of the actual malignant cases (Sensitivity = 0.9163) and correctly classifies 96.85% of the actual benign cases (Specificity = 0.9685).

```r
pred_probs <- predict(model_log, type = "response")


pred_class <- ifelse(pred_probs > 0.5, "malignant", "benign")
pred_class <- factor(pred_class, levels = c("benign", "malignant"))
conf_matrix <- table(Predicted = pred_class, Actual = biopsy$class)
print("Confusion Matrix:")
```

```
[1] "Confusion Matrix:"
```

```r
print(conf_matrix)
```

```
           Actual
Predicted   benign malignant
  benign       430        20
  malignant     14       219
```

```r
TP <- conf_matrix["malignant", "malignant"]
TN <- conf_matrix["benign", "benign"]
FP <- conf_matrix["malignant", "benign"]
FN <- conf_matrix["benign", "malignant"]

accuracy <- (TP + TN) / sum(conf_matrix)
sensitivity <- TP / (TP + FN) # True Positive Rate
specificity <- TN / (TN + FP) # True Negative Rate


cat("Accuracy:", round(accuracy, 4), "\n")
```

```
Accuracy: 0.9502
```

```r
cat("Sensitivity:", round(sensitivity, 4), "\n")
```

```
Sensitivity: 0.9163
```

```r
cat("Specificity:", round(specificity, 4), "\n")
```

```
Specificity: 0.9685
```

### (d)

For a new tumor observation with V1 = 5, V2 = 4, and V3 = 4, the model calculates a predicted conditional probability of approximately 0.666 (or 66.6%). Because this predicted probability of 0.666 exceeds the standard decision threshold of 0.5, the model formally classifies this specific tumor as malignant.

```
new_obs <- data.frame(V1 = 5, V2 = 4, V3 = 4)
pred_new <- predict(model_log, newdata = new_obs, type = "response")
print(pred_new)
```

```
        1
0.6660541
```

## Part 2

### 2.1

### (a)

Based on the df summary output showing 6 rows out of a full long-format matrix, the simulated dataset successfully established a completely crossed design where subjects (participant) and items (item) are properly recognized as nominal factors (), aligning with a foundational setup where every participant responds to multiple stimuli across distinct experimental blocks

```
library(lme4)
```

```
Loading required package: Matrix
```

```
Registered S3 method overwritten by 'lme4':
  method           from
  na.action.merMod car
```

```
set.seed(123)

n_participants <- 20
n_items <- 15
n_conditions <- 2
```

```r
df <- expand.grid(
  participant = factor(paste0("P", 1:n_participants)),
  item = factor(paste0("I", 1:n_items)),
  condition = factor(c("control", "treatment"), levels = c("control", "treatment"))
)


sigma_u0 <- 50
sigma_w0 <- 30
sigma_e  <- 40

sub_intercepts <- rnorm(n_participants, mean = 0, sd = sigma_u0)
names(sub_intercepts) <- paste0("P", 1:n_participants)

item_intercepts <- rnorm(n_items, mean = 0, sd = sigma_w0)
names(item_intercepts) <- paste0("I", 1:n_items)


beta_0 <- 400
beta_1 <- -30


df$rt <- beta_0 +
  ifelse(df$condition == "treatment", beta_1, 0) +
  sub_intercepts[df$participant] +
  item_intercepts[df$item] +
  rnorm(nrow(df), mean = 0, sd = sigma_e)

head(df)
```

```
  participant item condition       rt
1          P1   I1   control 367.4871
2          P2   I1   control 408.1127
3          P3   I1   control 371.0230
4          P4   I1   control 327.9347
5          P5   I1   control 442.0921
6          P6   I1   control 365.0695
```

**(b)**

The model selection metrics show that the OLS linear model (model_1) fits the data poorest with an AIC of 6756.000 and a BIC of 6769.191, whereas adding random intercepts for participants (model_2) reduces the AIC to 6410.577, and adding random intercepts for both participants and items (model_3) achieves the lowest values (AIC = 6228.438, BIC = 6250.423), proving that accounting for crossed dual-source variance drastically improves model fit.

```
model_1 <- lm(rt ~ condition, data = df)


model_2 <- lmer(rt ~ condition + (1 | participant), data = df)


model_3 <- lmer(rt ~ condition + (1 | participant) + (1 | item), data = df)


anova_comparison <- AIC(model_1, model_2, model_3)
print(" comparing AIC result ")
```

```
[1] " comparing AIC result "
```

```
print(anova_comparison)
```

```
        df      AIC
model_1  3 6756.000
model_2  4 6410.577
model_3  5 6228.438
```

```
print("comparing BIC result ")
```

```
[1] "comparing BIC result "
```

```
print(BIC(model_1, model_2, model_3))
```

```
        df      BIC
model_1  3 6769.191
model_2  4 6428.165
model_3  5 6250.423
```

### (c)

For the optimal model (model_3), the fixed effect intercept indicates that the baseline reaction time under the control condition is 405.66 ms, while the treatment significantly reduces reaction times by 32.81 ms ($t = -10.19, p < .001$). Furthermore, using the variance components ($\sigma^2_{\text{participant}} = 2351.12$, $\sigma^2_{\text{item}} = 769.93$, $\sigma^2_{\text{residual}} = 1554.33$), the participant-level ICC is calculated as 0.5029, meaning that approximately 50.29% of the unmodeled residual variation in reaction times is driven by stable, individual differences across participants.

```
summary_m3 <- summary(model_3)
print("Fixed effect estimates ")
```

```
[1] "Fixed effect estimates "
```

```
print(summary_m3$coefficients)
```

```
                   Estimate Std. Error   t value
(Intercept)        405.65535  13.193399  30.74684
conditiontreatment -32.81353   3.219036 -10.19359
```

```
variance_components <- as.data.frame(VarCorr(model_3))
print("Variance Component Details ")
```

```
[1] "Variance Component Details "
```

```
print(variance_components)
```

```
          grp        var1 var2      vcov    sdcor
1 participant (Intercept) <NA> 2351.1209 48.48836
2        item (Intercept) <NA>  769.9296 27.74761
3    Residual        <NA> <NA> 1554.3286 39.42497
```

```
var_participant <- variance_components$vcov[variance_components$grp == "participant"]
var_item        <- variance_components$vcov[variance_components$grp == "item"]
var_residual    <- variance_components$vcov[variance_components$grp == "Residual"]

icc_participant <- var_participant / (var_participant + var_item + var_residual)
cat("\nIntraclass correlation coefficient (ICC) of the subjects:", round(icc_participant, 4)
```

```
Intraclass correlation coefficient (ICC) of the subjects: 0.5029
```

### (d)

The likelihood ratio test comparing the random intercepts model (model_3) against the random slopes model (model_slope) yields a chi-squared statistic of $\chi^2(2) = 4.67$ with a corresponding $p$-value of 0.09698. Since this value exceeds the standard significance threshold ($\alpha = 0.05$), we fail to reject the null hypothesis, concluding that allowing the treatment effect to vary randomly across participants does not significantly improve the model beyond a simpler shared-slope structure.

```
model_slope <- lmer(rt ~ condition + (1 + condition | participant) + (1 | item),
                    data = df, REML = TRUE)


lrt_result <- anova(model_3, model_slope)
```

```
refitting model(s) with ML (instead of REML)
```

```
print("Likelihood Ratio Test (LRT) result ")
```

```
[1] "Likelihood Ratio Test (LRT) result "
```

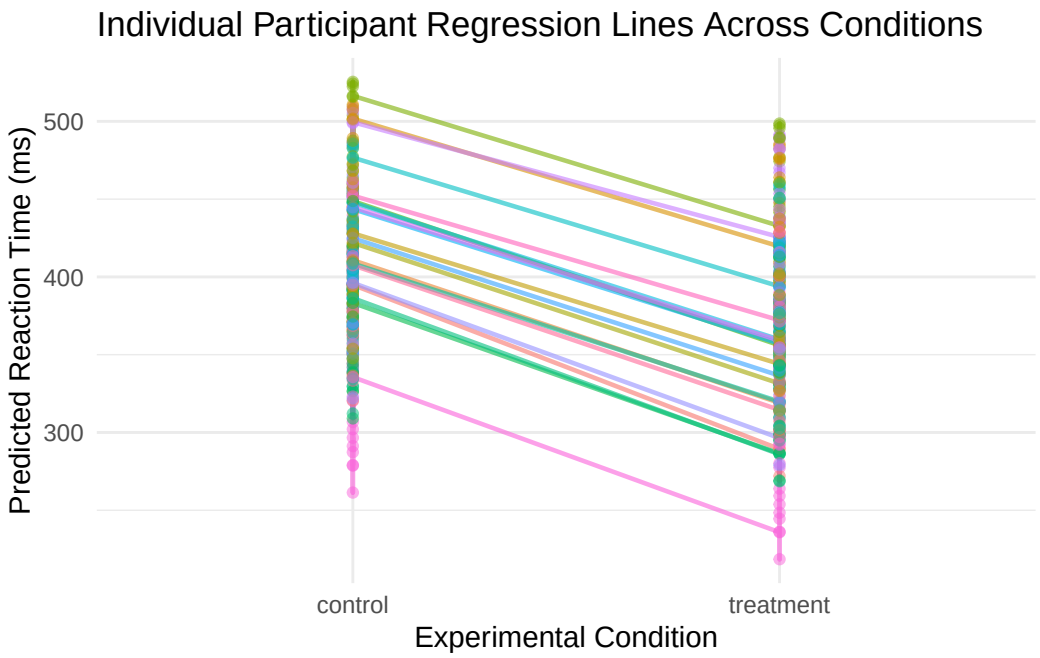
```
print(lrt_result)
```

```
Data: df
Models:
model_3: rt ~ condition + (1 | participant) + (1 | item)
model_slope: rt ~ condition + (1 + condition | participant) + (1 | item)
            npar    AIC    BIC  logLik -2*log(L)  Chisq Df Pr(>Chisq)
model_3        5 6239.6 6261.6 -3114.8    6229.6
model_slope    7 6238.9 6269.7 -3112.5    6224.9 4.6664  2    0.09698 .
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

### (e)

The generated ggplot visualization confirms that because the random slopes did not significantly deviate from a uniform effect across individuals, the reaction time slopes for all 20 participants follow a tight, downward-parallel trajectory moving from the control condition to the treatment condition, visually demonstrating that while individual baseline speeds differ, the behavioral reduction in latency is relatively uniform across the cohort

```r
library(ggplot2)


df$predicted_rt <- predict(model_slope)

ggplot(df, aes(x = condition, y = predicted_rt, group = participant, color = participant)) +
  geom_line(alpha = 0.6, linewidth = 0.8) +
  geom_point(alpha = 0.5) +
  labs(
    title = "Individual Participant Regression Lines Across Conditions",
    x = "Experimental Condition",
    y = "Predicted Reaction Time (ms)"
  ) +
  theme_minimal() +
  theme(legend.position = "none")
```



## Part 3

### 3.1

```
library(ggplot2)


theme_penguin <- function(base_size = 12, base_family = "sans") {

  theme_classic(base_size = base_size, base_family = base_family) %+replace%
    theme(

      plot.title = element_text(face = "bold", hjust = 0.5, size = rel(1.2), margin = margin


      axis.title.x = element_text(face = "bold", margin = margin(t = 10)),
      axis.title.y = element_text(face = "bold", margin = margin(r = 10), angle = 90),


      axis.text = element_text(color = "black", size = rel(0.9)),

      axis.line = element_line(linewidth = 0.8, color = "black"),


      legend.position = "bottom",
      legend.background = element_rect(color = "black", fill = "white", linewidth = 0.5),
      legend.title = element_text(face = "bold"),


      plot.margin = margin(15, 15, 15, 15)
    )
}
```

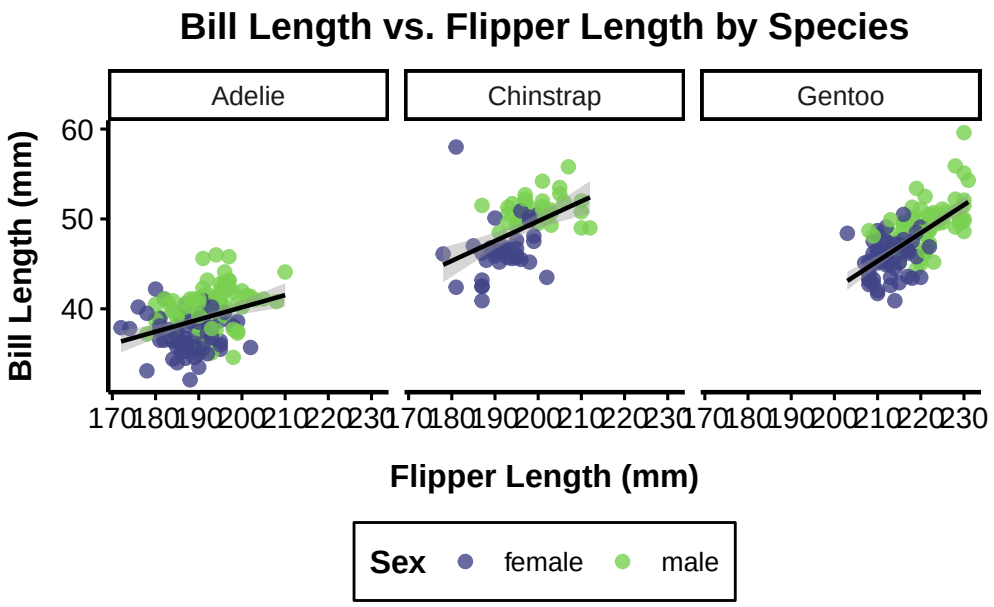
### 3.2

```
library(ggplot2)
library(palmerpenguins)
```

```
Attaching package: 'palmerpenguins'

The following objects are masked from 'package:datasets':

    penguins, penguins_raw
```

```r
penguins_clean <- subset(penguins, !is.na(sex))

ggplot(penguins_clean, aes(x = flipper_length_mm, y = bill_length_mm)) +

  geom_point(aes(color = sex), alpha = 0.8, size = 2) +

  geom_smooth(method = "lm", se = TRUE, color = "black", linewidth = 0.8) +

  facet_wrap(~ species) +

  scale_color_viridis_d(option = "D", begin = 0.2, end = 0.8) +

  labs(
    title = "Bill Length vs. Flipper Length by Species",
    x = "Flipper Length (mm)",
    y = "Bill Length (mm)",
    color = "Sex"
  ) +

  theme_penguin()
```

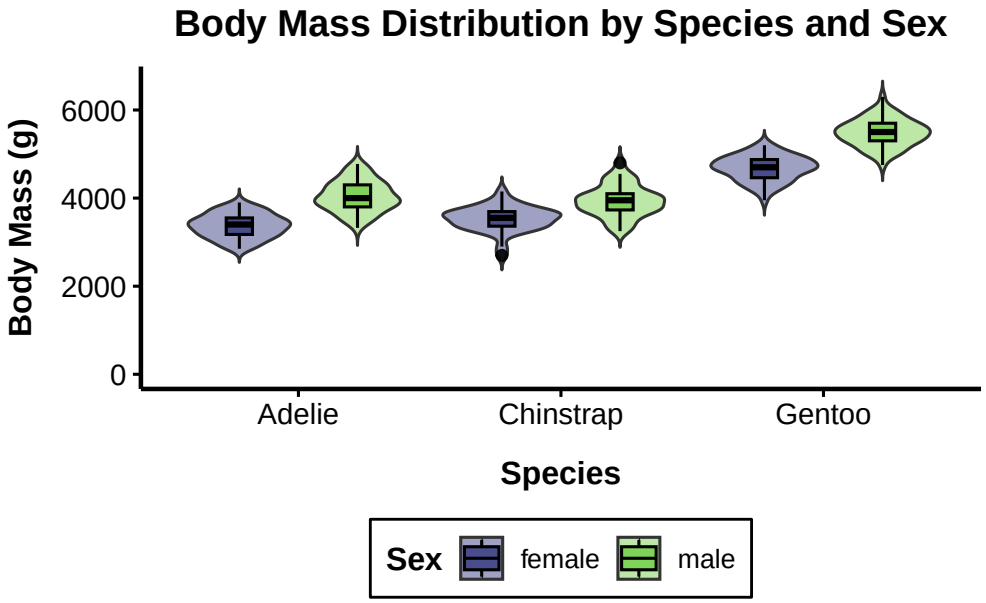
```
`geom_smooth()` using formula = 'y ~ x'
```




### 3.3

```
library(ggplot2)
library(palmerpenguins)


penguins_clean <- subset(penguins, !is.na(sex))


ggplot(penguins_clean, aes(x = species, y = body_mass_g, fill = sex)) +


  geom_violin(trim = FALSE, alpha = 0.5, position = position_dodge(0.9)) +

  geom_boxplot(width = 0.2, alpha = 0.9, position = position_dodge(0.9), color = "black") +


  scale_y_continuous(limits = c(0, NA)) +


  scale_fill_viridis_d(option = "D", begin = 0.2, end = 0.8) +
```

```
  labs(
    title = "Body Mass Distribution by Species and Sex",
    x = "Species",
    y = "Body Mass (g)",
    fill = "Sex"
  ) +


  theme_penguin()
```



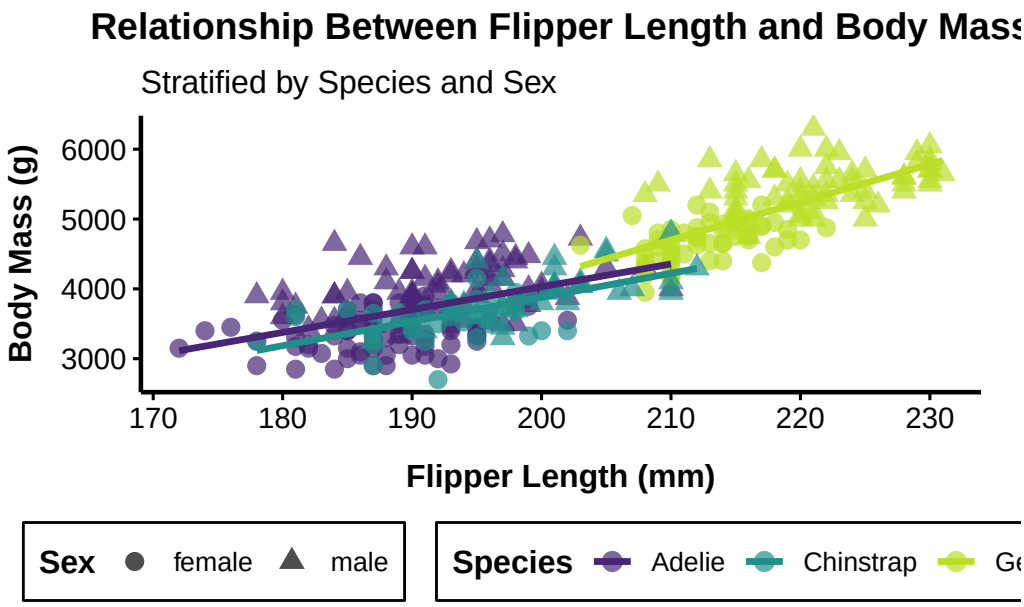

### 3.4

The visualization reveals a strong positive correlation between flipper length and body mass across all three penguin species, indicating that physically larger penguins consistently possess longer flippers. Furthermore, clear inter-species clustering is evident, with Gentoo penguins forming a distinct group characterized by significantly greater body mass and longer flippers compared to the closely clustered Adelie and Chinstrap penguins. Finally, sexual dimorphism is observable within each specific species cluster, as male penguins (represented by triangles) generally exhibit greater body mass and longer flippers than their female counterparts.

```
library(ggplot2)
library(palmerpenguins)
```

```r
penguins_clean <- na.omit(penguins)

ggplot(penguins_clean, aes(x = flipper_length_mm, y = body_mass_g, color = species)) +


  geom_point(aes(shape = sex), size = 3, alpha = 0.7) +


  geom_smooth(method = "lm", se = FALSE, linewidth = 1.2) +


  scale_color_viridis_d(option = "D", begin = 0.1, end = 0.9) +


  labs(
    title = "Relationship Between Flipper Length and Body Mass",
    subtitle = "Stratified by Species and Sex",
    x = "Flipper Length (mm)",
    y = "Body Mass (g)",
    color = "Species",
    shape = "Sex"
  ) +


  theme_penguin()
```

```
`geom_smooth()` using formula = 'y ~ x'
```




## Part 4

### (a)

```
library(shiny)
library(ggplot2)


ui <- fluidPage(
  titlePanel("mtcars Dataset Explorer"),

  sidebarLayout(
    sidebarPanel(

      selectInput(inputId = "predictor",
                  label = "Choose a predictor variable:",
                  choices = c("wt", "hp", "disp", "drat")),


    ),

    mainPanel(
```

```
      plotOutput(outputId = "scatter_plot"),
      verbatimTextOutput(outputId = "model_summary")
    )
  )
)
```

**(b)**

```
server <- function(input, output) {


  fit <- reactive({
    formula_str <- paste("mpg ~", input$predictor)
    lm(as.formula(formula_str), data = mtcars)
  })


  output$scatter_plot <- renderPlot({
    p <- ggplot(mtcars, aes(x = .data[[input$predictor]], y = mpg)) +
      geom_point(size = 3, alpha = 0.7) +
      labs(x = input$predictor, y = "Miles Per Gallon (mpg)") +
      theme_minimal()
        return(p)
  })


  output$model_summary <- renderPrint({
    summary(fit())
  })
}

shinyApp(ui = ui, server = server)
```

**(c)**

I chose to add a `checkboxInput()` to toggle the regression line. This allows users to visually assess the raw scatter of the data before imposing a linear fit, giving them dynamic visual control over the plot without cluttering the interface.

```r
library(shiny)
library(ggplot2)

# DESIGN CHOICE EXPLANATION FOR PART (c):
#I chose to add a `checkboxInput` to toggle the linear regression line along with its 95% co
# This allows users to flexibly inspect the raw data distribution before overlaying the stati
# Including the 95% confidence interval provides critical visual feedback regarding the model

ui <- fluidPage(
  titlePanel("mtcars Dataset Explorer"),

  sidebarLayout(
    sidebarPanel(

      selectInput(inputId = "predictor",
                  label = "Choose a predictor variable:",
                  choices = c("wt", "hp", "disp", "drat")),


      checkboxInput(inputId = "show_line",
                    label = "Show Regression Line with 95% CI",
                    value = TRUE)
    ),

    mainPanel(

      plotOutput(outputId = "scatter_plot"),
      verbatimTextOutput(outputId = "model_summary")
    )
  )
)


server <- function(input, output) {


  fit <- reactive({
    formula_str <- paste("mpg ~", input$predictor)
    lm(as.formula(formula_str), data = mtcars)
  })
```

```
  output$scatter_plot <- renderPlot({
    p <- ggplot(mtcars, aes(x = .data[[input$predictor]], y = mpg)) +
      geom_point(size = 3, alpha = 0.7) +
      labs(x = input$predictor, y = "Miles Per Gallon (mpg)") +
      theme_minimal()


    if (input$show_line) {
      p <- p + geom_smooth(method = "lm", se = TRUE, color = "blue")
    }

    return(p)
  })


  output$model_summary <- renderPrint({
    summary(fit())
  })
}


shinyApp(ui = ui, server = server)
```

# Part 5

## 5.1

### (a)

I defined safe_cores as logical_cores - 1. It is considered best practice to leave at least one core available for the operating system and background processes. If all available cores are allocated to a parallel computation, the entire system can become unresponsive or freeze until the task completes

```
library(parallel)


logical_cores <- detectCores(logical = TRUE)
cat("Logical Cores:", logical_cores, "\n")
```

```
Logical Cores: 10
```

```r
physical_cores <- detectCores(logical = FALSE)
cat("Physical Cores:", physical_cores, "\n")
```

```
Physical Cores: 10
```

```r
safe_cores <- logical_cores - 1
```

### (b)

The microbenchmark results clearly demonstrate the performance advantage of parallel computing. The median execution time for the sequential method (lapply) was 1014 units (approximately 1 second), which perfectly reflects the cumulative artificial delay of 0.01 seconds applied 100 times. In contrast, the parallel method (mclapply) significantly reduced the median time to 168 units. By distributing the independent tasks simultaneously across multiple cores, the parallel approach achieved an approximate 5.5x speed improvement compared to the sequential evaluation.

```r
library(microbenchmark)


slow_mean <- function(x) {
  Sys.sleep(0.01)
  mean(x)
}


set.seed(42)
random_vectors <- lapply(1:100, function(i) rnorm(50))


benchmark_results <- microbenchmark(
  sequential = lapply(random_vectors, slow_mean),
  parallel = mclapply(random_vectors, slow_mean, mc.cores = safe_cores),
  times = 10
)
```

```
Warning in microbenchmark(sequential = lapply(random_vectors, slow_mean), :
less accurate nanosecond times to avoid potential integer overflows
```

```r
print(benchmark_results)
```

```
Unit: milliseconds
       expr       min        lq      mean    median        uq       max neval
 sequential 1192.6979 1197.8578 1199.3524 1198.5379 1199.4466 1206.0498    10
   parallel  158.7768  165.7887  173.1254  170.9509  178.9358  195.9245    10
 cld
  a
   b
```

## 5.2

### (a)

```r
library(parallel)
library(microbenchmark)


bootstrap_ci_sequential <- function(data, n_bootstrap = 1000, confidence = 0.95) {

  boot_means <- unlist(lapply(1:n_bootstrap, function(i) {
    mean(sample(data, size = length(data), replace = TRUE))
  }))

  alpha <- 1 - confidence
  lower_bound <- quantile(boot_means, probs = alpha / 2)
  upper_bound <- quantile(boot_means, probs = 1 - (alpha / 2))


  return(c(lower = unname(lower_bound), upper = unname(upper_bound)))
}
```

### (b)

```r
bootstrap_ci_parallel <- function(data, n_bootstrap = 1000, confidence = 0.95) {

  boot_means <- unlist(mclapply(1:n_bootstrap, function(i) {
    mean(sample(data, size = length(data), replace = TRUE))
```

```
  }, mc.cores = safe_cores))


  alpha <- 1 - confidence
  lower_bound <- quantile(boot_means, probs = alpha / 2)
  upper_bound <- quantile(boot_means, probs = 1 - (alpha / 2))


  return(c(lower = unname(lower_bound), upper = unname(upper_bound)))
}
```

### (c)

The benchmark results reveal an important concept in parallel computing: overhead. For smaller n_bootstrap sizes (e.g., 500 and 1000), the sequential approach often outperforms the parallel approach. This is because the core task inside the loop—taking a small sample of 32 rows from mtcars$mpg and calculating its mean—is computationally trivial. The time required by the OS to create child processes, distribute the tasks across multiple cores, and collect the results (the parallel overhead) is significantly greater than the actual calculation time. As the problem size scales up to 2000 (and beyond), the computational workload eventually begins to offset the initial overhead cost, though for such simple vector operations, sequential execution remains highly efficient.

```
set.seed(228)


sizes <- c(500, 1000, 2000)


summary_table <- data.frame(
  n_bootstrap = integer(),
  sequential_time_ms = numeric(),
  parallel_time_ms = numeric()
)


for (n in sizes) {

  bm <- microbenchmark(
    sequential = bootstrap_ci_sequential(mtcars$mpg, n_bootstrap = n),
    parallel = bootstrap_ci_parallel(mtcars$mpg, n_bootstrap = n),
```

```
    times = 10
  )


  median_times <- summary(bm)$median


  summary_table <- rbind(summary_table, data.frame(
    n_bootstrap = n,
    sequential_time_ms = median_times[1],
    parallel_time_ms = median_times[2]
  ))
}


print(summary_table)
```

```
  n_bootstrap sequential_time_ms parallel_time_ms
1         500           5.491089         52.03851
2        1000          10.798457         38.60945
3        2000          20.244345         46.76608
```